

MoMo SMS Database Design

Team:

Devis Muhozi [d.muhozin@alustudent.com]

Samuel Hezekiah Epodoi [h.epodoi@alustudent.com]

Ivan Ineza Hakizimana [i.inezahak@alustudent.com]

Repository: <https://github.com/Devislastthought/momo-transaction-analytics>

Participation Task Sheet:

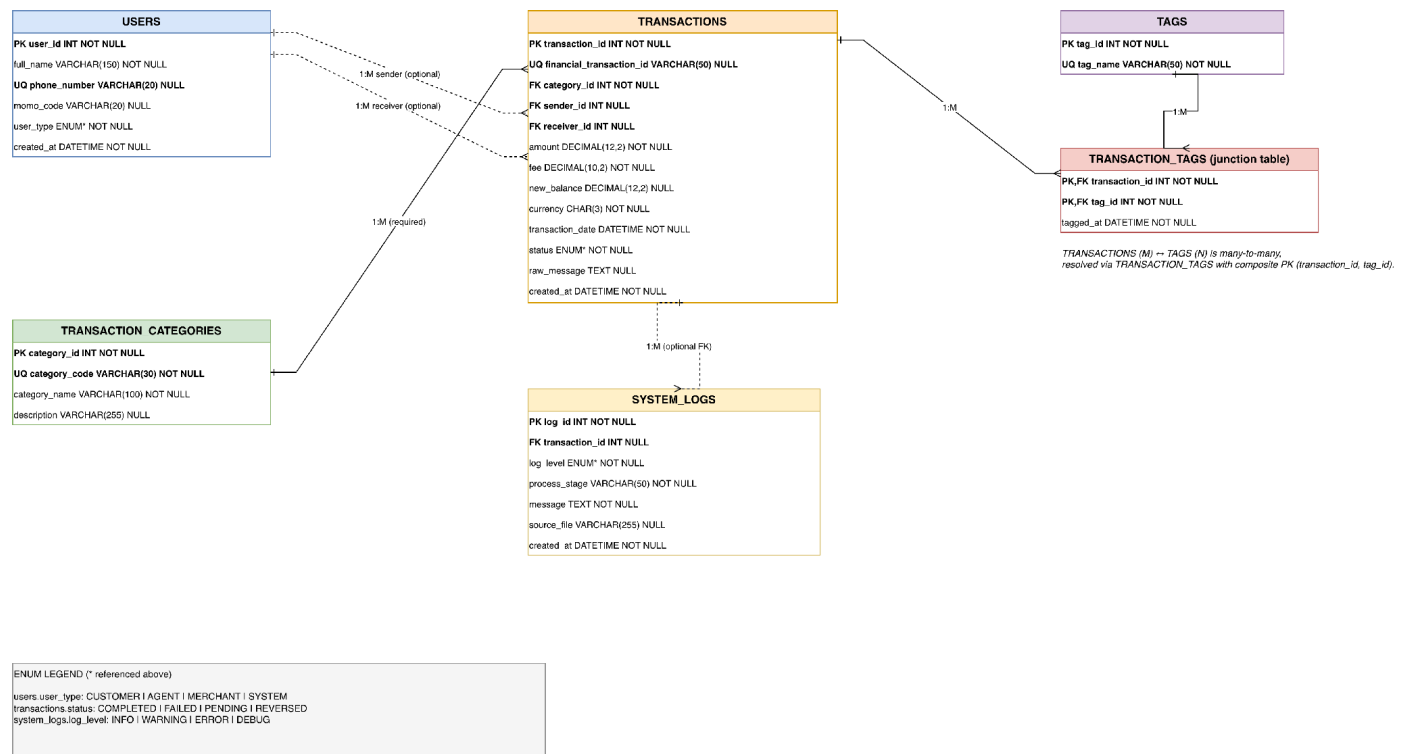
https://docs.google.com/spreadsheets/d/1GonHvdL1HM06K-z2PEIY-2wRZhMsZfxQQ4Bfkp_DD0/edit

Trello Board:

<https://trello.com/b/ne4syt27>

1. Entity relationship diagram

Source: docs/erd_diagram.drawio. Export: docs/erd_diagram.png.



2. Existing design rationale

Our ERD has six entities: USERS, TRANSACTION_CATEGORIES, TRANSACTIONS, TAGS, TRANSACTION_TAGS, and SYSTEM_LOGS. Each one traces back to something we actually found while going through modified_sms_v2.xml.

We pulled USERS out on its own because the same people keep showing up across many SMS messages. The same sender, the same agent, the same merchant. Instead of repeating a name and phone number on every transaction row, we store each person once and reference them by ID. user_type separates customers from agents and merchants, since the messages clearly distinguish agent withdrawals from merchant payments.

TRANSACTION_CATEGORIES exists because the SMS bodies fall into distinct, repeating patterns. We counted at least eight: incoming money, payments to a code holder, bank deposits, airtime, bundle purchases, MTN Cash Power, agent withdrawals, and third-party payments through ONAFRIQ. Keeping these as rows rather than hardcoded text means new categories can be added without touching the schema.

TRANSACTIONS is our central table. sender_id and receiver_id are two separate foreign keys back to USERS, since a person can appear as either depending on the message. We kept financial_transaction_id because the SMS TxId is a natural identifier separate from our own surrogate key, and raw_message so we can always trace a row back to its source SMS.

TAGS and TRANSACTION_TAGS resolve our required many to many relationship: a transaction can carry several free form labels, and a label can apply to many transactions. The junction table stores just the two foreign keys plus a timestamp, following the standard pattern for resolving M:N relationships.

SYSTEM_LOGS mirrors the ETL pipeline's own logging/dead letter behavior, so parsing failures and processing steps stay traceable even when no clean transaction record was produced.

3. Data dictionary (users)

Definitions extracted from database/database_setup.sql. Descriptions, where available, are copied from the column comments in Devis's shared archived SQL.

Column	SQL type	Null / default / key	Description
user_id	INT	NOT NULL; PK; auto increment	Surrogate key
full_name	VARCHAR(150)	NOT NULL	Name as it appears in the SMS body
phone_number	VARCHAR(20)	NULL allowed	MSISDN, may be partially masked in source SMS
momo_code	VARCHAR(20)	NULL allowed	Merchant/agent MoMo code, if applicable
user_type	ENUM('CUSTOMER','AGENT','MERCHANT','SYSTEM')	NOT NULL; default 'CUSTOMER'	Role this party plays in transactions
created_at	DATETIME	NOT NULL; default CURRENT_TIMESTAMP	

Additional constraints (from setup SQL)

```
CONSTRAINT uq_users_phone UNIQUE (phone_number)
CREATE INDEX idx_users_name ON users (full_name);
```

Data dictionary (transaction_categories)

Column	SQL type	Null / default / key	Description
category_id	INT	NOT NULL; PK; auto increment	
category_code	VARCHAR(30)	NOT NULL	Short machine code, e.g. RECEIVE_MONEY
category_name	VARCHAR(100)	NOT NULL	Human readable name
description	VARCHAR(255)	NULL allowed	

Additional constraints (from setup SQL)

```
CONSTRAINT uq_category_code UNIQUE (category_code)
```

Data dictionary (transactions)

Column	SQL type	Null / default / key	Description
transaction_id	INT	NOT NULL; PK; auto increment	
financial_transaction_id	VARCHAR(50)	NULL allowed	TxId extracted from the SMS body, when present
category_id	INT	NOT NULL	
sender_id	INT	NULL allowed	FK to users; NULL for inbound-only messages (e.g. bank deposit)
receiver_id	INT	NULL allowed	FK to users; NULL for outbound-only messages (e.g. withdrawal)

Column	SQL type	Null / default / key	Description
amount	DECIMAL(12,2)	NOT NULL	Transaction amount in currency units
fee	DECIMAL(10,2)	NOT NULL; default 0.00	
new_balance	DECIMAL(12,2)	NULL allowed	Balance reported after the transaction
currency	CHAR(3)	NOT NULL; default 'RWF'	
transaction_date	DATETIME	NOT NULL	Timestamp parsed from the SMS body
status	ENUM('COMPLETED','FAILED','PENDING','REVERSED')	NOT NULL; default 'COMPLETED'	
raw_message	TEXT	NULL allowed	Original SMS body, kept for audit/traceability
created_at	DATETIME	NOT NULL; default CURRENT_TIMESTAMP	

Additional constraints (from setup SQL)

```

CONSTRAINT uq_transactions_fin_id UNIQUE (financial_transaction_id)
CONSTRAINT fk_tx_category FOREIGN KEY (category_id) REFERENCES
transaction_categories(category_id)
ON UPDATE CASCADE ON DELETE RESTRICT
CONSTRAINT fk_tx_sender FOREIGN KEY (sender_id) REFERENCES users(user_id)
ON UPDATE CASCADE ON DELETE SET NULL
CONSTRAINT fk_tx_receiver FOREIGN KEY (receiver_id) REFERENCES users(user_id)
ON UPDATE CASCADE ON DELETE SET NULL
CONSTRAINT chk_amount_nonneg CHECK (amount >= 0)
CONSTRAINT chk_fee_nonneg CHECK (fee >= 0)
CREATE INDEX idx_tx_date ON transactions (transaction_date);
CREATE INDEX idx_tx_category ON transactions (category_id);
CREATE INDEX idx_tx_sender ON transactions (sender_id);
CREATE INDEX idx_tx_receiver ON transactions (receiver_id);
CREATE INDEX idx_tx_status ON transactions (status);

```

Data dictionary (tags)

Column	SQL type	Null / default / key	Description
tag_id	INT	NOT NULL; PK; auto increment	
tag_name	VARCHAR(50)	NOT NULL	

Additional constraints (from setup SQL)

```

CONSTRAINT uq_tag_name UNIQUE (tag_name)

```

Data dictionary (transaction_tags)

Column	SQL type	Null / default / key	Description
transaction_id	INT	NOT NULL	
tag_id	INT	NOT NULL	
tagged_at	DATETIME	NOT NULL; default CURRENT_TIMESTAMP	

Additional constraints (from setup SQL)

```

PRIMARY KEY (transaction_id, tag_id)
CONSTRAINT fk_tt_transaction FOREIGN KEY (transaction_id) REFERENCES
transactions(transaction_id)
ON UPDATE CASCADE ON DELETE CASCADE
CONSTRAINT fk_tt_tag FOREIGN KEY (tag_id) REFERENCES tags(tag_id)

```

ON UPDATE CASCADE ON DELETE CASCADE

Data dictionary (system_logs)

Column	SQL type	Null / default / key	Description
log_id	INT	NOT NULL; PK; auto increment	
transaction_id	INT	NULL allowed	Nullable: a parse failure may occur before a transaction row exists
log_level	ENUM('INFO','WARNING','ERROR','DEBUG')	NOT NULL; default 'INFO'	
process_stage	VARCHAR(50)	NOT NULL	e.g. parse_xml, clean_normalize, categorize, load_db
message	TEXT	NOT NULL	
source_file	VARCHAR(255)	NULL allowed	
created_at	DATETIME	NOT NULL; default CURRENT_TIMESTAMP	

Additional constraints (from setup SQL)

```
CONSTRAINT fk_log_transaction FOREIGN KEY (transaction_id) REFERENCES transactions(transaction_id)
```

```
ON UPDATE CASCADE ON DELETE SET NULL
```

```
CREATE INDEX idx_logs_level ON system_logs (log_level);
```

```
CREATE INDEX idx_logs_stage ON system_logs (process_stage);
```

4. Implementation and execution evidence

Recorded environment: MariaDB 10.4.27, Source distribution. Source:

docs/test-results/result_test_queries.txt. No MySQL engine execution transcript is present in the reviewed repository.

Setup script: database/database_setup.sql. It deletes and recreates momo_sms_db. Run it only in a disposable database environment or after backing up work that must be retained.

Table	Recorded baseline rows
users	6
transaction_categories	8
transactions	6
tags	5
transaction_tags	5
system_logs	5

Recorded final totals: 6 transactions; amount 71600.00; fees 200.00; zero remaining TEST-CRUD-001 rows.

Repository evidence files

docs/test-results/result_constraint_tests.txt

docs/test-results/result_database_setup.txt

docs/test-results/result_test_queries.txt

The setup transcript reports two warnings on DROP DATABASE; details were not recorded. The normal query transcript contains no errors. The constraint transcript records the seven expected errors.

5. CRUD and query results

CRUD statements from database/test_queries.sql

```
START TRANSACTION;
```

```
INSERT INTO transactions
```

```
(financial_transaction_id, category_id, sender_id, receiver_id,  
amount, fee, new_balance, transaction_date)
```

```
VALUES ('TEST-CRUD-001', 1, 2, 1, 5000.00, 0.00, 7000.00, '2026-09-15  
12:00:00');
```

```

SET @test_transaction_id = LAST_INSERT_ID();
SELECT transaction_id, financial_transaction_id, amount, status
FROM transactions WHERE transaction_id = @test_transaction_id;
UPDATE transactions SET status = 'REVERSED'
WHERE transaction_id = @test_transaction_id;
SELECT transaction_id, amount, status
FROM transactions WHERE transaction_id = @test_transaction_id;
DELETE FROM transactions WHERE transaction_id = @test_transaction_id;
SELECT COUNT(*) AS remaining_rows
FROM transactions WHERE transaction_id = @test_transaction_id;
ROLLBACK;

```

Recorded results: INSERT created transaction 7 for 5000.00 with COMPLETED status; UPDATE changed it to REVERSED; DELETE left zero matching rows. ROLLBACK completed. These are extracted observations from result_test_queries.txt.

Detail and summary queries

```

SELECT t.transaction_id, t.amount, t.currency, c.category_name,
       s.full_name AS sender, r.full_name AS receiver,
       GROUP_CONCAT(g.tag_name ORDER BY g.tag_name SEPARATOR ', ') AS tags
FROM transactions t
JOIN transaction_categories c ON c.category_id = t.category_id
LEFT JOIN users s ON s.user_id = t.sender_id
LEFT JOIN users r ON r.user_id = t.receiver_id
LEFT JOIN transaction_tags tt ON tt.transaction_id = t.transaction_id
LEFT JOIN tags g ON g.tag_id = tt.tag_id
GROUP BY t.transaction_id, t.amount, t.currency, c.category_name,
         s.full_name, r.full_name
ORDER BY t.transaction_id;

SELECT c.category_name, COUNT(t.transaction_id) AS transaction_count,
       COALESCE(SUM(t.amount), 0) AS total_amount
FROM transaction_categories c
LEFT JOIN transactions t ON t.category_id = c.category_id
GROUP BY c.category_id, c.category_name
ORDER BY c.category_id;

```

Recorded overall baseline amount: 71600.00 RWF; fees: 200.00 RWF. See the full transcript for detail rows and category totals.

6. Accuracy and security rules

Existing constraint tests and recorded outcomes

Operation	Existing rule	Recorded MariaDB outcome
Negative amount	chk_amount_nonneg	4025: rejected
Negative fee	chk_fee_nonneg	4025: rejected
Repeated financial ID	uq_transactions_fin_id	1062: rejected
Unknown category	fk_tx_category	1452: rejected
Repeated transaction/tag pair	Composite PRIMARY key	1062: rejected
Delete referenced category	fk_tx_category / RESTRICT	1451: rejected
NULL amount	amount NOT NULL	1048: rejected

Source: database/constraint_tests.sql and docs/test-results/result_constraint_tests.txt. MySQL error numbers may differ.

The normal query transcript also records zero junction links after deleting transaction 1, preservation of log 1 with a NULL transaction reference, and restoration after rollback.

7. JSON representations

Examples are in `examples/json_schemas.json`, with the nested response also in `examples/transaction_full.json`. The combined file covers all six tables. These are proposed API examples, not formal JSON Schema validation definitions.

Transaction 1 has a null receiver, matching SQL. Its sender, category, tags and logs are nested in the complete example. The raw SMS is included.

Insertion-time `created_at` and `tagged_at` values are omitted from these selected-field examples. `transaction_date` has no Z suffix because the seed SQL does not establish UTC. Mapping details are in `examples/README.md`.

8. Team review and submission

Participation sheet:

https://docs.google.com/spreadsheets/d/1GonHvdL1HM06K-z2PEIY-2wRZhMsZfxQQ4Bfkp_DD0/edit

AI Use

We confirm that AI was used as an assistive learning and verification tool rather than as a replacement for our own technical work. Our team used AI in a limited and supportive manner during this assignment. AI was not used to generate the database design, ERD structure, table relationships, SQL schema logic, or final technical explanations.

The main technical work was completed manually by the team through analysis of the provided MoMo SMS data, team discussions, database implementation, testing, and individual GitHub contributions. All important database design decisions were discussed and implemented by team members. Overall, the majority of the work submitted for this assignment was completed manually by the team, while AI played a limited role in syntax checking.